

Hardhat 3 for Blockchain Development

Brief Introduction, Major Commands, Usage, Workflow and Project Structure

1. What is Hardhat?

Hardhat is a development environment for building Ethereum and other EVM-compatible smart-contract applications. It provides tools for compiling Solidity contracts, running a local blockchain, testing and debugging contracts, deploying them, interacting with networks, and verifying deployed contracts. Hardhat 3 also supports both Solidity tests and TypeScript/JavaScript tests.

A typical Hardhat development workflow is: Write Solidity → Compile → Test → Start a local blockchain → Deploy → Interact → Deploy to a testnet → Verify.

Hardhat 3 requires a modern Node.js installation. The current Hardhat documentation should always be checked for the exact supported Node.js versions because requirements can change.

2. Major Hardhat 3 Commands

Command	Description / Purpose	Example Usage
npx hardhat --init	Create or initialize a Hardhat project.	npx hardhat --init
npx hardhat --help	Display available Hardhat commands/tasks and options.	npx hardhat --help
npx hardhat --version	Display the installed Hardhat version.	npx hardhat --version
npx hardhat build	Build the Hardhat project, including compilation-related build steps.	npx hardhat build
npx hardhat compile	Compile Solidity smart contracts and generate artifacts.	npx hardhat compile
npx hardhat clean	Delete generated build artifacts and cache so the project can be rebuilt cleanly.	npx hardhat clean
npx hardhat test	Run the project's configured tests.	npx hardhat test
npx hardhat test solidity	Run Solidity tests when Solidity testing is configured.	npx hardhat test solidity
npx hardhat node	Start a local Ethereum JSON-RPC development node.	npx hardhat node
npx hardhat console	Open the interactive Hardhat console.	npx hardhat console
npx hardhat console --network localhost	Open the Hardhat console connected to the local Hardhat node.	npx hardhat console --network localhost
npx hardhat ignition deploy	Deploy contracts using Hardhat Ignition.	npx hardhat ignition deploy ignition/modules/Counter.ts
npx hardhat ignition deploy ... --network localhost	Deploy an Ignition module to the local Hardhat node.	npx hardhat ignition deploy ignition/modules/Counter.ts --network localhost
npx hardhat ignition deploy ... --network sepolia	Deploy an Ignition module to the Sepolia test network.	npx hardhat ignition deploy ignition/modules/Counter.ts --network sepolia
npx hardhat verify	Verify a deployed contract when the appropriate verification plugin is installed.	npx hardhat verify <contract-address>
npx hardhat keystore set	Store a secret such as a private key or API key in the Hardhat keystore.	npx hardhat keystore set SEPOLIA_PRIVATE_KEY
npx hardhat help <task>	Display detailed help for a specific Hardhat task.	npx hardhat help test

3. Commands You Will Use Most Often

For a beginner, the following sequence covers the most common Hardhat development workflow.

```
npx hardhat --init
↓
npx hardhat compile
↓
npx hardhat test
↓
npx hardhat node
↓
npx hardhat ignition deploy ignition/modules/MyModule.ts --network localhost
↓
npx hardhat console --network localhost
↓
npx hardhat ignition deploy ignition/modules/MyModule.ts --network sepolia
```

4. Typical Hardhat 3 Workflow Diagram



5. Example: Compile and Test a Contract

After creating a contract such as `contracts/Calculator.sol`, compile the project:

```
npx hardhat compile
```

Then run the configured tests:

```
npx hardhat test
```

Hardhat 3 can be used with Solidity tests as well as TypeScript/JavaScript tests, depending on the project's plugins and configuration.

6. Start a Local Blockchain

Start a local Hardhat development blockchain in the first terminal:

```
npx hardhat node
```

Keep this terminal running. Open a second terminal and deploy the contract using Hardhat Ignition:

```
npx hardhat ignition deploy ignition/modules/Calculator.ts --network localhost
```

Then open an interactive console connected to the local blockchain:

```
npx hardhat console --network localhost
```

7. Hardhat Ignition

Hardhat Ignition is Hardhat's declarative deployment system. Instead of writing deployment logic as a sequence of manual steps, you define deployment modules. Ignition can track deployment state, recover from failures, resume interrupted deployments, and avoid unnecessary redeployment of contracts.

Example local deployment:

```
npx hardhat ignition deploy ignition/modules/Counter.ts --network localhost
```

Example Sepolia deployment:

```
npx hardhat ignition deploy ignition/modules/Counter.ts --network sepolia
```

8. Typical Hardhat 3 Project Structure

```
my-project/
├── contracts/
│   └── Counter.sol
├── test/
│   └── Counter.ts
├── ignition/
│   └── modules/
│       └── Counter.ts
├── scripts/
├── hardhat.config.ts
├── package.json
├── artifacts/
└── cache/
```

The contracts/ directory contains Solidity source files. The test/ directory contains tests. ignition/modules/ contains Ignition deployment modules. scripts/ is used for automation scripts. hardhat.config.ts is the central Hardhat configuration file. artifacts/ and cache/ are generated during compilation/build operations.

9. Important Hardhat 3 Note: Plugins and Toolboxes

Many Hardhat capabilities are provided through plugins and toolboxes. Therefore, two Hardhat 3 projects may expose slightly different tasks and commands depending on which plugins are installed. For your specific project, use 'npx hardhat --help' to see the authoritative list of available tasks.

```
npx hardhat --help
```

10. Recommended Beginner Learning Path

1. Create a Hardhat 3 project.
2. Create a simple Solidity contract such as MathsCalculator.sol.
3. Compile the smart contract.
4. Write and run tests.
5. Start the local Hardhat node.
6. Create an Ignition deployment module.
7. Deploy the contract to localhost.
8. Interact with the contract using the Hardhat console and Ethers.js or Viem, depending on the selected toolbox.
9. Configure the Sepolia test network.
10. Deploy the contract to Sepolia.
11. Connect MetaMask to the application/network as needed.
12. Verify the deployed contract using the appropriate verification plugin.

11. Useful Official Reference

Hardhat documentation: <https://hardhat.org/docs>

Note: Hardhat evolves rapidly. When a command differs from an older tutorial, prefer the output of 'npx hardhat --help' and the current Hardhat 3 documentation for the plugins installed in your project.